
Neural Networks by Hand

From Single-Neuron Intuition to Backpropagation and Matrix Calculus

All mathematical equations are typeset as native LaTeX in the source and PDF.

This guide is written for someone who wants to open the black box of neural networks and understand every calculation by hand. Rather than treating backpropagation as a mysterious set of matrix rules, we build the intuition step by step: how a single neuron weighs evidence, how layers compose simple curves into complex shapes, how the chain rule assigns credit or blame for an error, and how scalar derivatives assemble naturally into matrices.

Core mental model: A neural network is an assembly line of adjustable math operations. The **forward pass** feeds an input through the assembly line to produce a prediction. The **loss function** measures how far off that prediction is. **Backpropagation** walks backward down the assembly line to ask: *how much did each weight contribute to the mistake?* **Gradient descent** nudges each weight in the direction that reduces the mistake.

Learning goals By the end of this document, you should be able to look at equations such as

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f' \left(\mathbf{z}^{(l)} \right) \quad \text{and} \quad \nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T,$$

translate every symbol into plain English, explain why the matrix transpose $(W^{(l+1)})^T$ and outer product appear, and compute a complete forward and backward pass on paper with real numbers.

Contents

1	Training Objective and System Overview	4
1.1	Three Separate Jobs in Deep Learning	4
2	Artificial Neuron Mechanics	5
2.1	Intuitive Role of Weights	5
2.2	Intuitive Role of Bias	5
3	Activation Functions and Local Derivatives	6
3.1	Rectified Linear Unit (ReLU)	6
3.2	Sigmoid	6
3.3	Hyperbolic Tangent (Tanh)	6
4	Necessity of Nonlinearity	6
5	Layer Formulation and Matrix Shapes	7
5.1	Dimension Verification Rule	7
6	Forward Propagation	8
7	Loss Functions	8
8	Computational Graph and Chain Rule	8
9	Worked Single-Neuron Example	9
10	First-Order Taylor Justification for Subtracting the Gradient	10
11	Vectorized Backpropagation and Credit Assignment	10
12	Intuitive Derivation of the Matrix Equations	11
12.1	Weight Gradient Outer Product Structure	11
12.2	Hidden Recurrence Matrix Transpose	11
13	Worked Two-Layer Network by Hand	12
13.1	Step-by-Step Forward Pass	12
13.2	Step-by-Step Backward Pass	13
14	Jacobian Matrices and Reverse-Mode Efficiency	13
15	Softmax and Cross-Entropy Classification	13

15.1	Logit-to-Probability Mapping (Softmax)	14
15.2	Cross-Entropy Loss and Derivative Cancellation	14
16	Mini-Batches and Parameter Optimizers	14
17	Gradient Verification and Deep Network Pathologies	15
17.1	Centered Finite-Difference Verification	15
17.2	Vanishing and Exploding Gradients	15
18	Practice Exercise and Complete Worked Solution	15
18.1	Problem Statement	16
18.2	Complete Step-by-Step Solution	16
19	Notation and Equation Reference	17

1 Training Objective and System Overview

Before diving into derivatives and matrices, it helps to see the big picture. A neural network is not a program written with explicit **if/else** rules. Instead, it is a mathematical formula with thousands or millions of adjustable knobs called **parameters** (weights and biases), collected into a single symbol θ .

Given an input vector $\mathbf{x}$ (for example, pixel intensities of an image or measurements from a sensor) and current parameter settings θ , the network computes a prediction $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}} = f(\mathbf{x}; \theta).$$

At the start of training, the knobs θ are set to random numbers, so the prediction $\hat{\mathbf{y}}$ is mostly nonsense. To teach the network, we compare its prediction $\hat{\mathbf{y}}$ against the true answer $\mathbf{y}$ using a **loss function**:

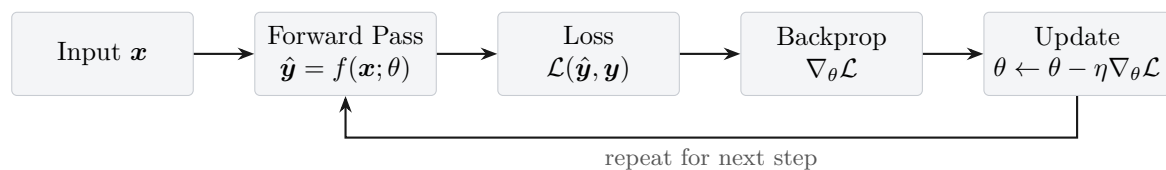
$$\mathcal{L} = \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}).$$

The loss $\mathcal{L}$ is a single number (a scalar) that acts as an error thermometer:

- A **large loss** means the prediction $\hat{\mathbf{y}}$ is far from the target $\mathbf{y}$.
- A **small loss** means the prediction is close to the target.

Training is the search for parameter settings θ^* that make this loss as small as possible:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f(\mathbf{x}; \theta), \mathbf{y}).$$



Analogy (Tuning a mixing console): Imagine a studio mixing console with 10,000 unmarked sliders (θ), and a single digital meter on the wall ($\mathcal{L}$) showing how distorted the audio is (0 = perfect). You cannot see inside the wiring, and moving one slider changes how other sliders behave. Calculus gives you a superpower: in a single pass, it tells you the exact slope of the meter with respect to every slider on the board. That list of slopes is the **gradient** $\nabla_{\theta} \mathcal{L}$.

1.1 Three Separate Jobs in Deep Learning

Beginners often blur three distinct concepts together. Keep them separate at all times:

1. **Forward propagation (prediction):** Given input $\mathbf{x}$ and current parameters θ , compute the output $\hat{\mathbf{y}}$ and the loss $\mathcal{L}$.
2. **Backpropagation (blame assignment):** Given the loss $\mathcal{L}$, use the chain rule in reverse to compute the exact sensitivity $\nabla_{\theta} \mathcal{L}$ of the loss to every parameter.
3. **Optimization (parameter update):** Use the gradient $\nabla_{\theta} \mathcal{L}$ to adjust the parameters (for example, via SGD or Adam) so the next prediction has lower loss.

2 Artificial Neuron Mechanics

A neural network is built by connecting many simple computational units called **neurons**. Despite the biological name, an artificial neuron is just a two-step mathematical formula:

1. **Step 1 (Linear weighted sum):** Multiply each input by a weight, add them all up, and add a bias.
2. **Step 2 (Nonlinear activation):** Pass that sum through a shaping function (such as ReLU or sigmoid).

Suppose a neuron receives n input numbers $\mathbf{x}$ and has n corresponding weights $\mathbf{w}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}.$$

First, the neuron computes its **pre-activation** z :

$$z = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \sum_{i=1}^n w_i x_i + b.$$

Second, the neuron applies a scalar **activation function** f to z to produce its output a (called the **activation**):

$$a = f(z) = f(\mathbf{w}^T \mathbf{x} + b).$$

Complete equation of a single neuron:

$$z = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z).$$

Always write a neuron as these two sub-steps ($x \rightarrow z \rightarrow a$). When we differentiate later, splitting the linear step (z) from the nonlinear step (a) makes the calculus effortless.

2.1 Intuitive Role of Weights

Think of the inputs $x_1, \dots, x_n$ as pieces of evidence, and the weights $w_1, \dots, w_n$ as how much the neuron trusts or cares about each piece of evidence:

- **Positive weight** ($w_i > 0$): Input x_i is *excitatory*. Increasing x_i pushes the neuron's score z upward.
- **Negative weight** ($w_i < 0$): Input x_i is *inhibitory*. Increasing x_i pulls the neuron's score z downward.
- **Zero weight** ($w_i = 0$): The neuron completely ignores input x_i .

2.2 Intuitive Role of Bias

What does the bias b do? Suppose we have one input x , so $z = wx + b$. Without b , $z = wx$ is forced to equal 0 whenever $x = 0$. Graphically, a line $z = wx$ is pinned to pass through the origin $(0, 0)$.

Adding b shifts the line up or down (or shifts the decision boundary left or right). You can think of $-b$ as the neuron's **activation threshold**: how much weighted evidence $\mathbf{w}^T \mathbf{x}$ is needed before the pre-activation z crosses zero.

3 Activation Functions and Local Derivatives

During the forward pass, the activation function $f(z)$ shapes the neuron's output. During the backward pass, its derivative $f'(z)$ acts as a **local gatekeeper** that scales gradients passing backward through the neuron.

3.1 Rectified Linear Unit (ReLU)

ReLU is the default activation function in modern deep learning because of its simplicity: if the input z is negative, output 0; if z is positive, pass z through unchanged.

$$f(z) = \max(0, z) = \begin{cases} 0, & z < 0, \\ z, & z \geq 0, \end{cases} \quad f'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

Intuition for backpropagation: ReLU acts like an open/closed switch. When $z > 0$, the neuron is active and its derivative is 1, so gradients pass backward through it at 100% strength without shrinking. When $z < 0$, the neuron is off and its derivative is 0, completely blocking gradient flow along that path.

3.2 Sigmoid

The logistic sigmoid squashes any real number $z \in (-\infty, \infty)$ smoothly into the probability range $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a).$$

Why this derivative formula is so convenient: Notice that once you have computed the forward output $a = \sigma(z)$, you already know the derivative! If a sigmoid neuron outputs $a = 0.8$, its slope is immediately $a(1 - a) = 0.8 \times 0.2 = 0.16$.

Important catch: When z is very positive ($a \approx 1$) or very negative ($a \approx 0$), the curve flattens out and $a(1 - a) \approx 0$. Even at its steepest point ($z = 0$, where $a = 0.5$), the maximum possible slope of a sigmoid is only $0.5 \times 0.5 = 0.25$.

3.3 Hyperbolic Tangent (Tanh)

Tanh is a rescaled sigmoid that outputs values centered around zero in $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{d}{dz} \tanh(z) = 1 - \tanh^2(z) = 1 - a^2.$$

At $z = 0$, $a = 0$ and the slope is $1 - 0^2 = 1$. Like sigmoid, its slope approaches 0 when $|z|$ is large ($a \approx \pm 1$).

4 Necessity of Nonlinearity

Why do we bother putting nonlinear activation functions $f(z)$ between layers at all? What happens if we stack multiple layers of matrix multiplications and biases without any activation function?

Suppose Layer 1 computes $\mathbf{h} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$ and Layer 2 directly computes $\mathbf{y} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$. Substituting $\mathbf{h}$ into Layer 2 gives:

$$\begin{aligned}\mathbf{y} &= W^{(2)}(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= \underbrace{(W^{(2)}W^{(1)})}_{W'}\mathbf{x} + \underbrace{(W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})}_{\mathbf{b}'} \\ &= W'\mathbf{x} + \mathbf{b}'.\end{aligned}$$

Because multiplying two matrices $W^{(2)}W^{(1)}$ just produces another single matrix W' , a 100-layer linear network is mathematically identical to a 1-layer linear network! It can still only draw flat lines or flat hyperplanes. Inserting nonlinear activations (f) bends the space between layers so the network can approximate curved, complex functions.

5 Layer Formulation and Matrix Shapes

Instead of writing separate equations for every neuron in a layer, we stack all neurons of layer l into a single matrix-vector equation.

Suppose layer $l - 1$ outputs a column vector $\mathbf{a}^{(l-1)}$ with n_{l-1} numbers, and layer l contains n_l neurons. Each of the n_l neurons in layer l has its own row of weights (of length n_{l-1}) and its own scalar bias. Stacking those weight rows into a matrix $W^{(l)}$ gives:

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}),$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the raw input vector.

How to read a weight matrix $W^{(l)}$ without getting confused:

- Each **row** j of $W^{(l)}$ belongs to **one destination neuron j in the current layer l** .
- Each **column** i of $W^{(l)}$ corresponds to **one source neuron i from the previous layer $l - 1$** .
- Therefore, the entry $W_{ji}^{(l)}$ is the weight on the connection **from unit i (in layer $l - 1$) to unit j (in layer l)**.

This explains the shape rule immediately: $W^{(l)}$ always has shape (destination size) $\times$ (source size) $= n_l \times n_{l-1}$.

5.1 Dimension Verification Rule

When working by hand, always write the matrix shapes underneath your equation:

$$\underbrace{\mathbf{z}^{(l)}}_{(n_l \times 1)} = \underbrace{W^{(l)}}_{(n_l \times n_{l-1})} \underbrace{\mathbf{a}^{(l-1)}}_{(n_{l-1} \times 1)} + \underbrace{\mathbf{b}^{(l)}}_{(n_l \times 1)}.$$

Notice how the inner dimensions (n_{l-1}) match and cancel in the matrix product $(n_l \times n_{l-1})(n_{l-1} \times 1) \rightarrow (n_l \times 1)$. In backpropagation, the gradient of a scalar loss $\mathcal{L}$ with respect to **any** matrix or vector must have the **exact same dimensions** as that matrix or vector:

$$\nabla_{W^{(l)}} \mathcal{L} \in \mathbb{R}^{n_l \times n_{l-1}}, \quad \nabla_{\mathbf{b}^{(l)}} \mathcal{L} \in \mathbb{R}^{n_l \times 1}, \quad \boldsymbol{\delta}^{(l)} = \nabla_{\mathbf{z}^{(l)}} \mathcal{L} \in \mathbb{R}^{n_l \times 1}.$$

If your derived formula for $\nabla_{W^{(l)}} \mathcal{L}$ does not have shape $n_l \times n_{l-1}$, you know immediately that a transpose is missing or in the wrong place.

6 Forward Propagation

Forward propagation simply means evaluating the layer equations in order from $l = 1$ to $l = L$, saving the intermediate vectors $\mathbf{z}^{(l)}$ and $\mathbf{a}^{(l)}$ along the way because we will need them during the backward pass.

For a two-layer network ($L = 2$):

$$\begin{aligned}\mathbf{z}^{(1)} &= W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} && \text{(Layer 1 linear combination)} \\ \mathbf{a}^{(1)} &= f^{(1)}(\mathbf{z}^{(1)}) && \text{(Layer 1 hidden activations)} \\ \mathbf{z}^{(2)} &= W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} && \text{(Layer 2 linear combination)} \\ \hat{\mathbf{y}} = \mathbf{a}^{(2)} &= f^{(2)}(\mathbf{z}^{(2)}) && \text{(Network prediction)}\end{aligned}$$

Written as one nested function:

$$\hat{\mathbf{y}} = f^{(2)}\left(W^{(2)}f^{(1)}\left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}\right).$$

Because the output is a function inside a function inside a function, differentiating it with respect to $W^{(1)}$ or $W^{(2)}$ is a textbook application of the **chain rule**.

7 Loss Functions

For a scalar prediction a and scalar target y , the most common regression loss is **squared error**:

$$\mathcal{L} = \frac{1}{2}(a - y)^2.$$

Why do we include the $\frac{1}{2}$ in front? Because when we differentiate $(a - y)^2$ with respect to a , the exponent 2 comes down and cancels the $\frac{1}{2}$:

$$\frac{\partial \mathcal{L}}{\partial a} = \frac{1}{2} \cdot 2(a - y) = a - y = \text{prediction} - \text{target}.$$

Look at how intuitive $\frac{\partial \mathcal{L}}{\partial a} = a - y$ is:

- If $a = 5$ and $y = 3$, the error derivative is $+2$ (prediction is 2 units too high).
- If $a = 1$ and $y = 3$, the error derivative is -2 (prediction is 2 units too low).
- If $a = 3$ and $y = 3$, the error derivative is 0 (prediction is exact).

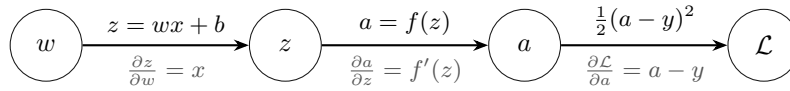
For a vector output $\mathbf{a} \in \mathbb{R}^K$ and target $\mathbf{y} \in \mathbb{R}^K$, we sum the squared errors across all components:

$$\mathcal{L} = \frac{1}{2} \sum_{i=1}^K (a_i - y_i)^2 = \frac{1}{2} \|\mathbf{a} - \mathbf{y}\|_2^2, \quad \nabla_{\mathbf{a}} \mathcal{L} = \mathbf{a} - \mathbf{y}.$$

8 Computational Graph and Chain Rule

To see how backpropagation works without getting lost in indices, let us trace a single neuron from start to finish:

$$z = wx + b, \quad a = f(z), \quad \mathcal{L} = \frac{1}{2}(a - y)^2.$$



Suppose we wiggle w by a tiny amount Δw . How does $\mathcal{L}$ change?

1. Wiggling w changes z at rate $\frac{\partial z}{\partial w} = x$.
2. Wiggling z changes a at rate $\frac{\partial a}{\partial z} = f'(z)$.
3. Wiggling a changes $\mathcal{L}$ at rate $\frac{\partial \mathcal{L}}{\partial a} = a - y$.

The chain rule says that the total amplification from w to $\mathcal{L}$ is the product of the local rates along the path:

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{\frac{\partial \mathcal{L}}{\partial a}}_{\text{output error}} \cdot \underbrace{\frac{\partial a}{\partial z}}_{\text{activation slope}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{\text{input strength}} = (a - y) f'(z) x.$$

Notice that the first two factors together measure how sensitive the loss is to the neuron's pre-activation z . We give that combined quantity a special name, the **error signal** δ :

$$\delta \equiv \frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial z} = (a - y) f'(z).$$

Once we have δ , the derivatives for the weight w and bias b are immediate:

Single-neuron gradient pattern (the heart of backpropagation):

$$\delta = (a - y) f'(z), \quad \frac{\partial \mathcal{L}}{\partial w} = \delta \cdot x, \quad \frac{\partial \mathcal{L}}{\partial b} = \delta \cdot 1 = \delta.$$

Plain-English rule: The gradient on any connection weight is always equal to the error signal δ at the destination neuron multiplied by the activation x entering from the source neuron: (destination error δ) $\times$ (source activation x).

9 Worked Single-Neuron Example

Let us put real numbers into a single sigmoid neuron so every step is concrete. Suppose:

$$x = 2, \quad w = 0.5, \quad b = 0.2, \quad y = 1, \quad f(z) = \sigma(z).$$

Step 1: Forward pass (compute prediction and loss):

$$\begin{aligned} z &= wx + b = (0.5)(2) + 0.2 = 1.2, \\ a &= \sigma(1.2) = \frac{1}{1 + e^{-1.2}} \approx 0.7685, \\ \mathcal{L} &= \frac{1}{2}(a - y)^2 = \frac{1}{2}(0.7685 - 1)^2 \approx 0.0267. \end{aligned}$$

Interpretation: The target is $y = 1$, and our neuron predicted $a = 0.7685$ (too low by 0.2315).

Step 2: Backward pass (compute sensitivities from right to left):

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial a} &= a - y = 0.7685 - 1 = -0.2315 && \text{(Prediction is too low } \Rightarrow \text{ negative error)} \\
 \frac{\partial a}{\partial z} &= a(1 - a) \approx (0.7685)(0.2315) \approx 0.1779 && \text{(Local slope of sigmoid at } z = 1.2) \\
 \delta = \frac{\partial \mathcal{L}}{\partial z} &= (-0.2315)(0.1779) \approx -0.0412 && \text{(Error signal at pre-activation } z) \\
 \frac{\partial \mathcal{L}}{\partial w} &= \delta \cdot x = (-0.0412)(2) \approx -0.0824 && \text{(Weight gradient)} \\
 \frac{\partial \mathcal{L}}{\partial b} &= \delta \cdot 1 \approx -0.0412 && \text{(Bias gradient)}
 \end{aligned}$$

Step 3: Sign verification: Both $\partial \mathcal{L} / \partial w = -0.0824$ and $\partial \mathcal{L} / \partial b = -0.0412$ are negative because the prediction $a = 0.7685$ was *below* the target $y = 1$, and the input $x = +2$ is positive. A negative derivative means “*increasing w will decrease the loss.*” Indeed, when gradient descent subtracts a negative gradient ($w \leftarrow w - \eta(-0.0824)$), w increases, pushing the prediction closer to 1.

10 First-Order Taylor Justification for Subtracting the Gradient

For any smooth loss function $\mathcal{L}(\theta)$ and a small parameter change $\Delta\theta$, a first-order Taylor expansion approximates how the loss will change:

$$\mathcal{L}(\theta + \Delta\theta) \approx \mathcal{L}(\theta) + \nabla_{\theta} \mathcal{L}(\theta)^T \Delta\theta.$$

We want $\mathcal{L}(\theta + \Delta\theta) < \mathcal{L}(\theta)$, which means we want the dot product $\nabla_{\theta} \mathcal{L}(\theta)^T \Delta\theta$ to be negative. If we choose our step directly opposite to the gradient,

$$\Delta\theta = -\eta \nabla_{\theta} \mathcal{L}(\theta) \quad (\eta > 0),$$

then the dot product becomes a sum of squares:

$$\mathcal{L}(\theta - \eta \nabla_{\theta} \mathcal{L}(\theta)) \approx \mathcal{L}(\theta) - \eta \nabla_{\theta} \mathcal{L}(\theta)^T \nabla_{\theta} \mathcal{L}(\theta) = \mathcal{L}(\theta) - \eta \|\nabla_{\theta} \mathcal{L}(\theta)\|_2^2.$$

Because the squared length $\|\nabla_{\theta} \mathcal{L}(\theta)\|_2^2$ is strictly positive whenever the gradient is nonzero, stepping in the direction $-\nabla_{\theta} \mathcal{L}$ is guaranteed to decrease the loss for a sufficiently small step size η .

11 Vectorized Backpropagation and Credit Assignment

Now we scale up from one neuron to an entire multi-layer network. At every layer l , we define the **error vector** $\delta^{(l)} \in \mathbb{R}^{n_l}$ as the sensitivity of the loss to each neuron’s pre-activation $z_j^{(l)}$:

$$\delta^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \begin{bmatrix} \partial \mathcal{L} / \partial z_1^{(l)} \\ \vdots \\ \partial \mathcal{L} / \partial z_{n_l}^{(l)} \end{bmatrix}.$$

Once you know $\delta^{(l)}$ for every layer, you know all the gradients in the network. Backpropagation is a four-equation recipe for computing $\delta^{(L)}, \delta^{(L-1)}, \dots, \delta^{(1)}$ from the output layer back to the first layer.

Core Backpropagation Equations:

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot f'^{(L)}(\mathbf{z}^{(L)}) \quad (1. \text{ Output layer error signal}) \quad (1)$$

$$\delta^{(l)} = \left(\left(W^{(l+1)} \right)^T \delta^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)}) \quad (2. \text{ Hidden layer error recurrence}) \quad (2)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T \quad (3. \text{ Weight matrix gradient}) \quad (3)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \quad (4. \text{ Bias vector gradient}) \quad (4)$$

Here $\odot$ denotes element-wise (Hadamard) multiplication of two vectors of the same length.

12 Intuitive Derivation of the Matrix Equations

Equation (3) uses an outer product $\delta^{(l)}(\mathbf{a}^{(l-1)})^T$, and Equation (2) multiplies by the transpose $(W^{(l+1)})^T$. Both follow directly from the single-neuron chain rule.

12.1 Weight Gradient Outer Product Structure

Recall from our single-neuron rule that the gradient on a single wire $W_{ji}^{(l)}$ connecting source unit i (with activation $a_i^{(l-1)}$) to destination unit j (with error signal $\delta_j^{(l)}$) is just their product:

$$\frac{\partial \mathcal{L}}{\partial W_{ji}^{(l)}} = \delta_j^{(l)} a_i^{(l-1)}.$$

To form a matrix whose (j, i) entry is $\delta_j a_i$ for every pair (j, i) simultaneously, we multiply the column vector $\delta^{(l)} \in \mathbb{R}^{n_l \times 1}$ by the row vector $(\mathbf{a}^{(l-1)})^T \in \mathbb{R}^{1 \times n_{l-1}}$:

$$\delta^{(l)} \left(\mathbf{a}^{(l-1)} \right)^T = \begin{bmatrix} \delta_1 \\ \delta_2 \\ \vdots \\ \delta_{n_l} \end{bmatrix} \begin{bmatrix} a_1 & a_2 & \cdots & a_{n_{l-1}} \end{bmatrix} = \begin{bmatrix} \delta_1 a_1 & \delta_1 a_2 & \cdots & \delta_1 a_{n_{l-1}} \\ \delta_2 a_1 & \delta_2 a_2 & \cdots & \delta_2 a_{n_{l-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \delta_{n_l} a_1 & \delta_{n_l} a_2 & \cdots & \delta_{n_l} a_{n_{l-1}} \end{bmatrix}.$$

The outer product $\delta^{(l)}(\mathbf{a}^{(l-1)})^T$ is a multiplication table of every destination error δ_j with every source input a_i .

12.2 Hidden Recurrence Matrix Transpose

Now consider a hidden neuron j in layer l . In the forward pass, its output $a_j^{(l)}$ fans out and feeds into *every* neuron $k \in \{1, \dots, n_{l+1}\}$ in the next layer via the weights $W_{kj}^{(l+1)}$:

$$z_k^{(l+1)} = \sum_{j=1}^{n_l} W_{kj}^{(l+1)} a_j^{(l)} + b_k^{(l+1)}.$$

Because $a_j^{(l)}$ contributed to all n_{l+1} neurons in the next layer, it shares responsibility for all of their errors $\delta_1^{(l+1)}, \dots, \delta_{n_{l+1}}^{(l+1)}$. Each next-layer error $\delta_k^{(l+1)}$ is weighted by the connection strength

$W_{kj}^{(l+1)}$ along which $a_j^{(l)}$ influenced it:

$$\frac{\partial \mathcal{L}}{\partial a_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)}.$$

Visualizing the transpose $(W^{(l+1)})^T$:

- In the **forward pass**, signals travel from layer l to layer $l + 1$, so we multiply by $W^{(l+1)}$ (shape $n_{l+1} \times n_l$).
- In the **backward pass**, error signals travel in the opposite direction—from layer $l + 1$ back to layer l —along the *exact same wires*. Reversing the direction of every wire swaps sources and destinations, which in matrix algebra is the **transpose** $(W^{(l+1)})^T$ (shape $n_l \times n_{l+1}$).

Finally, to pass from the post-activation sensitivity $\partial \mathcal{L} / \partial \mathbf{a}^{(l)}$ back across the activation gate to the pre-activation $\mathbf{z}^{(l)}$, we multiply element-wise by the local slope $f'^{(l)}(\mathbf{z}^{(l)})$:

$$\boldsymbol{\delta}^{(l)} = \left((W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)}(\mathbf{z}^{(l)}).$$

13 Worked Two-Layer Network by Hand

Let us trace a complete $2 \rightarrow 2 \rightarrow 1$ network (2 inputs, 2 hidden ReLU neurons, 1 linear output neuron) with squared error loss.

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.4 & -0.6 \end{bmatrix}, \quad b^{(2)} = 0.2, \quad y = 0.5.$$

13.1 Step-by-Step Forward Pass

1. Hidden layer pre-activation $\mathbf{z}^{(1)}$ (shape 2×1):

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.5 & -0.2 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.5 - 0.4 + 0.1 \\ 0.3 + 1.6 - 0.1 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}.$$

2. Hidden layer ReLU activation $\mathbf{a}^{(1)}$ (shape 2×1): Since both $0.2 > 0$ and $1.8 > 0$, both hidden neurons are in the active regime of ReLU:

$$\mathbf{a}^{(1)} = \max(\mathbf{0}, \mathbf{z}^{(1)}) = \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix}, \quad f'^{(1)}(\mathbf{z}^{(1)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

3. Output layer prediction $\hat{y}$ and loss $\mathcal{L}$ (scalars):

$$\begin{aligned} \hat{y} = z^{(2)} &= W^{(2)}\mathbf{a}^{(1)} + b^{(2)} = \begin{bmatrix} 0.4 & -0.6 \end{bmatrix} \begin{bmatrix} 0.2 \\ 1.8 \end{bmatrix} + 0.2 = 0.08 - 1.08 + 0.2 = -0.8, \\ \mathcal{L} &= \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(-0.8 - 0.5)^2 = \frac{1}{2}(-1.3)^2 = 0.845. \end{aligned}$$

13.2 Step-by-Step Backward Pass

1. Output error signal $\delta^{(2)}$ (shape 1×1): Because the output layer is linear ($f'^{(2)}(z^{(2)}) = 1$), the output error signal is simply the prediction error:

$$\delta^{(2)} = \hat{y} - y = -0.8 - 0.5 = -1.3.$$

2. Layer 2 weight and bias gradients: Multiply destination error $\delta^{(2)}$ by the incoming hidden activations $(\mathbf{a}^{(1)})^T$:

$$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)} (\mathbf{a}^{(1)})^T = -1.3 \begin{bmatrix} 0.2 & 1.8 \end{bmatrix} = \begin{bmatrix} -0.26 & -2.34 \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)} = -1.3.$$

Intuition check: Notice that the second weight gradient (-2.34) is $9\times$ larger than the first weight gradient (-0.26) because hidden unit 2 ($a_2^{(1)} = 1.8$) was $9\times$ more active than hidden unit 1 ($a_1^{(1)} = 0.2$).

3. Propagate error backward to Hidden Layer 1 ($\delta^{(1)}$, shape 2×1): Send $\delta^{(2)} = -1.3$ backward through the transposed output weights $(W^{(2)})^T$:

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)} \right) \odot f'^{(1)}(\mathbf{z}^{(1)}) = \begin{bmatrix} 0.4 \\ -0.6 \end{bmatrix} (-1.3) \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}.$$

Intuition check: Why did hidden unit 1 get a negative error signal (-0.52) while hidden unit 2 got a positive error signal ($+0.78$)? Because hidden unit 1 has a positive weight ($+0.4$) to the output (so increasing it helps raise $\hat{y}$ toward $+0.5$), whereas hidden unit 2 has a negative weight (-0.6) to the output (so hidden unit 2 was pulling $\hat{y}$ down and needs to be reduced!).

4. Layer 1 weight and bias gradients: Form the outer product of $\delta^{(1)}$ with the input $\mathbf{x}^T = [1, 2]$:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)} \mathbf{x}^T = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} -0.52 & -1.04 \\ 0.78 & 1.56 \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \delta^{(1)} = \begin{bmatrix} -0.52 \\ 0.78 \end{bmatrix}.$$

14 Jacobian Matrices and Reverse-Mode Efficiency

For any vector function $\mathbf{y} = f(\mathbf{x})$ mapping $\mathbb{R}^n \rightarrow \mathbb{R}^m$, the **Jacobian matrix** $J_f \in \mathbb{R}^{m \times n}$ collects all partial derivatives $(J_f)_{ij} = \partial y_i / \partial x_j$. For a composition $\mathbf{y} = f(g(\mathbf{x}))$, the multivariable chain rule multiplies the Jacobians:

$$J_{f \circ g} = J_f J_g.$$

Why does backpropagation work *backward* from the scalar loss $\mathcal{L}$ instead of forward from the inputs?

- A neural network maps many parameters (millions) to a **single scalar loss** $\mathcal{L} \in \mathbb{R}^1$.
- Starting from the scalar output $\mathcal{L}$, the sensitivity $\mathbf{v} = \nabla_{\mathbf{y}} \mathcal{L}$ is just a single vector. Multiplying a vector by each layer's transposed Jacobian ($\nabla_{\mathbf{x}} \mathcal{L} = J_g^T (J_f^T \mathbf{v})$) is a series of cheap **vector-Jacobian products (VJPs)** rather than massive matrix-matrix multiplications.
- This allows reverse-mode differentiation (backpropagation) to compute the gradient with respect to *all* millions of parameters in roughly the same time as a single forward pass.

15 Softmax and Cross-Entropy Classification

When a network classifies an input into one of K categories, its final linear layer produces K raw scores called **logits** $\mathbf{z} = [z_1, \dots, z_K]^T$.

15.1 Logit-to-Probability Mapping (Softmax)

Exponentiating each logit ($e^{z_i} > 0$) makes all scores positive, and dividing by their sum forces them to add up to 1:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = \frac{e^{z_i-m}}{\sum_{j=1}^K e^{z_j-m}}, \quad m = \max_j z_j.$$

Subtracting the maximum logit m before exponentiating leaves the probabilities mathematically identical while preventing floating-point overflow ($e^{1000} = \infty$) on a computer.

15.2 Cross-Entropy Loss and Derivative Cancellation

If the true class is represented as a one-hot vector $\mathbf{y}$ (where $y_c = 1$ for the correct class c and 0 for all others), **cross-entropy loss** measures how much probability the model assigned to the correct class:

$$\mathcal{L} = - \sum_{i=1}^K y_i \log p_i = -\log p_c.$$

If the model assigns $p_c = 1.0$ to the true class, $\mathcal{L} = -\log(1) = 0$. If $p_c \rightarrow 0$, $\mathcal{L} \rightarrow \infty$.

When we substitute the softmax formula into cross-entropy and differentiate with respect to logit z_k , the logarithms and exponentials cancel cleanly:

$$\mathcal{L} = - \sum_{i=1}^K y_i z_i + \log \left(\sum_{j=1}^K e^{z_j} \right) \implies \frac{\partial \mathcal{L}}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = p_k - y_k.$$

Softmax + Cross-Entropy Output Error Signal:

$$\delta^{(L)} = \nabla_{\mathbf{z}^{(L)}} \mathcal{L} = \mathbf{p} - \mathbf{y}.$$

Intuition: If the true label for a 3-class problem is $\mathbf{y} = [0, 1, 0]^T$ and your network predicts probabilities $\mathbf{p} = [0.1, 0.7, 0.2]^T$, the output error signal is immediately $\delta^{(L)} = [0.1, -0.3, 0.2]^T$ —no extra matrix derivatives required.

16 Mini-Batches and Parameter Optimizers

In practice, we evaluate the network on a **mini-batch** of m examples at a time. Because differentiation is linear, the gradient of the average batch loss is simply the average of the per-example gradients:

$$\mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \mathcal{L}_i \implies \nabla_{\theta} \mathcal{L}_{\text{batch}} = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} \mathcal{L}_i.$$

Once backpropagation provides the batch gradient $\mathbf{g}_t = \nabla_{\theta} \mathcal{L}_{\text{batch}}(\theta_t)$, an optimizer updates the parameters:

- **Stochastic Gradient Descent (SGD):** Takes a direct step proportional to the current gradient:

$$\theta_{t+1} = \theta_t - \eta \mathbf{g}_t.$$

- **Momentum:** Keeps a moving average velocity $\mathbf{v}_t$ of past gradients so consistent directions build speed while zig-zagging oscillations cancel out:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \mathbf{v}_t.$$

- **RMSProp:** Divides each parameter's step by the root-mean-square of its recent gradients, giving steep coordinates smaller steps and flat coordinates larger steps:

$$\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}.$$

- **Adam (Adaptive Moment Estimation):** Combines Momentum (first moment $\mathbf{m}_t$) and RMSProp (second moment $\mathbf{v}_t$) with startup bias correction:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \\ \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, & \theta_{t+1} &= \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \end{aligned}$$

17 Gradient Verification and Deep Network Pathologies

17.1 Centered Finite-Difference Verification

Whenever you derive or code backpropagation by hand, you can check every single derivative using basic arithmetic. Nudge one parameter θ_i up and down by a tiny number ϵ (such as $\epsilon = 10^{-5}$) and measure how the loss changes:

$$g_{\text{numeric}} = \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}.$$

If your analytical backprop gradient g_{analytic} matches g_{numeric} to 5+ decimal places, your backpropagation derivation is guaranteed to be bug-free.

17.2 Vanishing and Exploding Gradients

In an L -layer network, computing the gradient at Layer 1 requires multiplying L layer Jacobians and activation derivatives together:

$$\boldsymbol{\delta}^{(1)} = \left(\prod_{l=1}^{L-1} \text{diag}(f'^{(l)}(\mathbf{z}^{(l)})) \left(\mathbf{W}^{(l+1)} \right)^T \right) \boldsymbol{\delta}^{(L)}.$$

- **Vanishing gradients:** If each layer's factor has magnitude < 1 (for instance, sigmoid derivatives are at most 0.25, and $0.25^{10} \approx 10^{-6}$), the error signal $\boldsymbol{\delta}^{(1)}$ reaching early layers shrinks to nearly zero. Early layers stop learning completely.
- **Exploding gradients:** If weight matrices have singular values > 1 (for instance, $2^{20} \approx 10^6$), gradients grow exponentially backward and cause numerical overflow (NaN).

18 Practice Exercise and Complete Worked Solution

18.1 Problem Statement

Consider a $2 \rightarrow 2 \rightarrow 1$ network with ReLU hidden activation, linear output, squared error loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$, learning rate $\eta = 0.1$, and target $y = 0.8$:

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 0.4 & -0.7 \\ 0.2 & 0.5 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.6 & -0.3 \end{bmatrix}, \quad b^{(2)} = 0.1.$$

Compute the forward pass, backward pass, and one gradient-descent update by hand. Try solving it on paper before reading the solution below.

18.2 Complete Step-by-Step Solution

1. Forward Pass:

$$\begin{aligned} \mathbf{z}^{(1)} &= W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 0.4(2) + (-0.7)(-1) + 0.1 \\ 0.2(2) + 0.5(-1) - 0.2 \end{bmatrix} = \begin{bmatrix} 0.8 + 0.7 + 0.1 \\ 0.4 - 0.5 - 0.2 \end{bmatrix} = \begin{bmatrix} 1.6 \\ -0.3 \end{bmatrix}, \\ \mathbf{a}^{(1)} &= \max(\mathbf{0}, \mathbf{z}^{(1)}) = \begin{bmatrix} 1.6 \\ 0 \end{bmatrix}, \quad f'^{(1)}(\mathbf{z}^{(1)}) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (\text{unit 2 inactive: } z_2^{(1)} = -0.3 < 0), \\ \hat{y} = z^{(2)} &= W^{(2)}\mathbf{a}^{(1)} + b^{(2)} = (0.6)(1.6) + (-0.3)(0) + 0.1 = 0.96 + 0.1 = 1.06, \\ \mathcal{L} &= \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(1.06 - 0.8)^2 = \frac{1}{2}(0.26)^2 = 0.0338. \end{aligned}$$

2. Backward Pass:

$$\begin{aligned} \delta^{(2)} &= \hat{y} - y = 1.06 - 0.8 = +0.26 \quad (\text{prediction overshoot target by } 0.26), \\ \nabla_{W^{(2)}}\mathcal{L} &= \delta^{(2)} (\mathbf{a}^{(1)})^T = 0.26 \begin{bmatrix} 1.6 & 0 \end{bmatrix} = \begin{bmatrix} 0.416 & 0 \end{bmatrix}, \quad \nabla_{b^{(2)}}\mathcal{L} = 0.26, \\ \boldsymbol{\delta}^{(1)} &= \left((W^{(2)})^T \delta^{(2)} \right) \odot f'^{(1)}(\mathbf{z}^{(1)}) = \begin{bmatrix} 0.6 \\ -0.3 \end{bmatrix} (0.26) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.156 \\ -0.078 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.156 \\ 0 \end{bmatrix}, \\ \nabla_{W^{(1)}}\mathcal{L} &= \boldsymbol{\delta}^{(1)} \mathbf{x}^T = \begin{bmatrix} 0.156 \\ 0 \end{bmatrix} \begin{bmatrix} 2 & -1 \end{bmatrix} = \begin{bmatrix} 0.312 & -0.156 \\ 0 & 0 \end{bmatrix}, \quad \nabla_{\mathbf{b}^{(1)}}\mathcal{L} = \begin{bmatrix} 0.156 \\ 0 \end{bmatrix}. \end{aligned}$$

Key observation: Because hidden neuron 2 had a negative pre-activation ($z_2^{(1)} = -0.3$), its ReLU gate was closed ($a_2^{(1)} = 0$ and $f'(z_2^{(1)}) = 0$). As a result, its outgoing weight $W_{12}^{(2)}$ and all of its incoming weights in row 2 of $W^{(1)}$ received **zero gradient** on this example.

3. Parameter Update ($\eta = 0.1$):

$$\begin{aligned} W_{\text{new}}^{(2)} &= \begin{bmatrix} 0.6 & -0.3 \end{bmatrix} - 0.1 \begin{bmatrix} 0.416 & 0 \end{bmatrix} = \begin{bmatrix} 0.5584 & -0.3 \end{bmatrix}, \quad b_{\text{new}}^{(2)} = 0.1 - 0.1(0.26) = 0.074, \\ W_{\text{new}}^{(1)} &= \begin{bmatrix} 0.4 & -0.7 \\ 0.2 & 0.5 \end{bmatrix} - 0.1 \begin{bmatrix} 0.312 & -0.156 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.3688 & -0.6844 \\ 0.2 & 0.5 \end{bmatrix}, \quad \mathbf{b}_{\text{new}}^{(1)} = \begin{bmatrix} 0.0844 \\ -0.2 \end{bmatrix}. \end{aligned}$$

19 Notation and Equation Reference

Symbol / Operation	Formula / Shape	Intuitive Meaning
Input $\mathbf{x} = \mathbf{a}^{(0)}$	$\mathbb{R}^{n_0 \times 1}$	Raw input features fed into the network
Weight Matrix $W^{(l)}$	$\mathbb{R}^{n_l \times n_{l-1}}$	Connection strengths from layer $l - 1$ to layer l
Bias Vector $\mathbf{b}^{(l)}$	$\mathbb{R}^{n_l \times 1}$	Per-neuron offset / activation threshold shift
Pre-Activation $\mathbf{z}^{(l)}$	$W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \in \mathbb{R}^{n_l \times 1}$	Weighted sum entering layer l 's activation
Activation $\mathbf{a}^{(l)}$	$f^{(l)}(\mathbf{z}^{(l)}) \in \mathbb{R}^{n_l \times 1}$	Nonlinear output of layer l passed to $l + 1$
Scalar Loss $\mathcal{L}$	$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) \in \mathbb{R}$	Single number measuring prediction error
Error Signal $\boldsymbol{\delta}^{(l)}$	$\partial \mathcal{L} / \partial \mathbf{z}^{(l)} \in \mathbb{R}^{n_l \times 1}$	Sensitivity of loss to layer l 's pre-activations
Hidden Recurrence	$((W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}) \odot f'^{(l)}(\mathbf{z}^{(l)})$	Pulls next-layer errors backward along wires
Weight Gradient $\nabla_{W^{(l)}} \mathcal{L}$	$\boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T \in \mathbb{R}^{n_l \times n_{l-1}}$	(Destination error $\boldsymbol{\delta}^{(l)}$) $\times$ (Source input $\mathbf{a}^{(l-1)}$)
Bias Gradient $\nabla_{\mathbf{b}^{(l)}} \mathcal{L}$	$\boldsymbol{\delta}^{(l)} \in \mathbb{R}^{n_l \times 1}$	Equal to the destination error signal $\boldsymbol{\delta}^{(l)}$
Parameter Update	$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$	Nudges parameters downhill to reduce loss